

MANUALES TÉCNICOS Y DE USUARIO

Intérprete e IDE del Lenguaje GoLite

Estudiante: Jhostin Javier Ramírez Enríquez

Carnet: 202404407

Curso: Organización de Lenguajes y Compiladores 1

Fecha: Junio 2026

Institución: Universidad de San Carlos de Guatemala

Facultad: Facultad de Ingeniería - Escuela de Ciencias y Sistemas

1. Manual de Usuario

| 1.1 Introducción al Entorno GoLite IDE

El Entorno de Desarrollo Integrado (IDE) de GoLite es una herramienta de escritorio robusta diseñada específicamente para facilitar la edición, el análisis y la ejecución estructurada del lenguaje GoLite. Gracias a su interfaz intuitiva basada en pestañas, los estudiantes y programadores pueden gestionar múltiples archivos de código fuente de manera simultánea.

| 1.2 Requisitos del Sistema

- Java Desarrollo Kit (JDK) versión 17 o superior instalado.
- Sistema Operativo: Windows 10/11, macOS o Linux compatible con entornos de escritorio Java Swing/JavaFX.
- Resolución de pantalla mínima de 1024x768 píxeles.

| 1.3 Componentes Principales de la Interfaz

Área del Editor

Ubicada en la sección central superior. Permite escribir el código fuente y organiza múltiples archivos abiertos mediante un sistema de pestañas independientes (por ejemplo, `main.glt` y `funciones.glt`). Muestra dinámicamente indicadores visuales y filas de numeración.

Consola de Salida

Posicionada en la parte inferior del entorno. Es un componente crítico que muestra en tiempo real todas las notificaciones del ciclo de vida del proceso de interpretación, mensajes informativos del sistema ([INFO] Compilación iniciada), impresiones de datos estándar del usuario mediante comandos nativos y reportes resumidos del intérprete.

| 1.4 Guía Práctica de Funcionalidades

Gestión de Archivos (Menú Archivo)

- **Crear archivos:** Genera un espacio de edición en blanco dentro de una pestaña nueva.
- **Abrir archivos:** Carga archivos del disco duro con extensión oficial `.glt` directamente hacia el editor de texto.
- **Guardar archivos:** Persiste los cambios actuales del búfer activo en el almacenamiento local en un archivo estructurado con terminación compatible.

Ejecución de Programas

Para iniciar el análisis del archivo actual, el usuario simplemente debe presionar el botón **"Ejecutar"** ubicado de forma prominente en la barra de herramientas superior. Esto desencadenará la secuencia automatizada

del analizador léxico, sintáctico y semántico, interpretando consecutivamente la lógica codificada a partir de su función inicial `func main()`.

Visualización de Reportes

A través del menú dedicado "**Reportes**", el usuario puede exportar y examinar los siguientes documentos interactivos en formato HTML/Visual:

- **Reporte de Errores:** Desglosa de manera concisa cada fallo léxico, sintáctico o semántico, indicando su tipo exacto, línea y columna correspondiente.
- **Tabla de Tokens:** Muestra la secuencia exacta de componentes léxicos válidos detectados.
- **Tabla de Símbolos:** Cataloga todas las variables, funciones y estructuras inicializadas junto con sus respectivos entornos de ámbito y tipos estáticos asociados.
- **Reporte de Árbol Sintáctico (AST):** Despliega un diagrama jerárquico del procesamiento interno.

2. Manual Técnico

2.1 Arquitectura del Intérprete

El motor del lenguaje GoLite se compone de un canal de procesamiento por etapas clásico para intérpretes con tipado estático, construido sobre la plataforma de desarrollo **Java**.

El flujo arquitectónico consta de las siguientes fases secuenciales obligatorias:

- Fase Léxica (JFLEX):** Transforma el flujo continuo de caracteres del archivo fuente `.glt` en un conjunto lógico de Tokens válidos, descartando elementos no operativos como comentarios de una línea (`//`) o bloques multilinea (`/* ... */`).
- Fase Sintáctica (CUP):** Recibe la secuencia ordenada de tokens y valida la correcta jerarquía y orden de instrucciones mediante una gramática LALR. Construye como salida el Árbol de Sintaxis Abstracta (AST).
- Fase Semántica:** Recorre el AST para poblar la Tabla de Símbolos, verificar la consistencia de tipos estáticos, el aislamiento de ámbitos de bloques (`{ }`) y la correcta definición de referencias de funciones y estructuras.
- Fase de Interpretación:** Ejecuta secuencialmente los nodos del AST empezando por la rutina `func main()`.

2.2 Especificación del Sistema de Tipos

GoLite implementa un control estricto de tipos estáticos. No se permite bajo ninguna circunstancia reasignar valores de naturalezas incompatibles a variables pre-declaradas. Los tipos primitivos aceptados y sus valores iniciales automatizados por defecto son:

Tipo Primitivo	Definición Operativa	Valor por Defecto
int	Valores numéricos de tipo entero (32 bits)	0
float64	Números de punto flotante de alta precisión (64 bits)	0.0
string	Cadenas de texto encerradas en comillas dobles ("")	"" (cadena vacía)
bool	Estructuras lógicas de control booleano (true, false)	false
rune	Alias compacto para representación de caracteres individuales (' ')	0

Nota de Implementación: Cualquier tipo compuesto (como `slice` o `struct`) que se inicialice sin valores explícitos toma el valor nulo universal por defecto representado mediante la palabra clave reservada `nil`.

| 2.3 Componentes y Lógica Estructurada

Variables y Ámbitos

Las variables pueden definirse explícitamente usando la nomenclatura `var id tipo = expresion` o de forma implícita y abreviada por medio del token operador `id := expresion`. Cada bloque delimitado por llaves define un espacio o entorno local de aislamiento semántico.

Estructuras Compuestas Controladas

- **Slices:** Matrices dinámicas ordenadas con soporte multidimensional y pasadas nativamente por referencia a través de parámetros. Soportan funciones embebidas como `len()`, `append()`, y utilidades avanzadas como `slices.Index` o `strings.Join`.
- **Structs:** Tipos de datos personalizados definidos por el usuario en el ámbito global. Sus propiedades internas se acceden o mutan a través del operador de acceso de punto (`instancia.atributo`). Sus funciones asociadas operan obligatoriamente bajo paso por referencia.

| 2.4 Manejo de Errores y Excepciones

El sistema cuenta con un recolector centralizado de errores para evitar que la aplicación colapse súbitamente durante incidentes semánticos o sintácticos. Los fallos se agrupan en una lista global estructurada basada en objetos con propiedades de ubicación explícitas:

```
public class ErrorCompilador {  
    private String tipo;          // Léxico, Sintáctico o Semántico  
    private int linea;           // Ubicación exacta en fila  
    private int columna;         // Ubicación exacta en columna  
    private String descripcion;  // Detalle explícito del fallo  
}
```